BÀI TẬP KỸ THUẬT HỘP TRẮNG

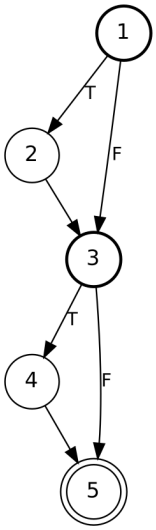
Sinh viên: **Trần Nhựt Anh** · MSSV: **22130015**

Bài 1. Thuật toán tìm số lớn nhất trong 3 số a, b, c

```
1. public int Max(int a, int b, int c) {
2.     max = a;
3.     if (max < b)
4.         max = b;
5.     if (max < c)
6.         max = c;
7.     return max;
8. }
```

1) Đồ thị luồng điều khiển (CFG)

Nút	Dòng lệnh	Nội dung
1	2–3	max = a; if (max < b) — nút quyết định
2	4	max = b
3	5	if (max < c) — nút quyết định
4	6	max = c
5	7	return max (nút kết thúc)



Chú thích: T = điều kiện đúng, F = điều kiện sai; nút viền đậm là nút quyết định. Số cạnh E = 6, số nút N = 5 ⇒ V(G) = E − N + 2 = 3 (= 2 nút quyết định + 1).

2) Test case đạt bao phủ nhánh 100%

Có 4 nhánh cần phủ: 1→2 (max<b đúng), 1→3 (sai), 3→4 (max<c đúng), 3→5 (sai).

TC	a	b	c	Nhánh được phủ	Đường đi	Kết quả mong đợi
TC1	1	2	3	1→2, 3→4	1-2-3-4-5	3
TC2	3	2	1	1→3, 3→5	1-3-5	3

Kết luận: 2 test case phủ đủ 4/4 nhánh ⇒ bao phủ nhánh 100% (đồng thời phủ 100% câu lệnh). Nếu muốn phủ cả 4 đường đi có thể thêm TC3: (1, 3, 2) → 3 (đường 1-2-3-5) và TC4: (3, 1, 4) → 4 (đường 1-3-4-5).

Bài 2. Lưu đồ giải phương trình bậc 2

Các điểm quyết định của lưu đồ: **a = 0**, **delta < 0**, **delta = 0**. Lưu đồ không có vòng lặp nên có đúng **4 đường đi** từ Begin đến End:

Đường	Mô tả
Đ1	a = 0 (đúng) → Print error → End
Đ2	a = 0 (sai) → delta < 0 (đúng) → Print: no results → End
Đ3	a = 0 (sai) → delta < 0 (sai) → delta = 0 (đúng) → Print −b/(2a) → End
Đ4	a = 0 (sai) → delta < 0 (sai) → delta = 0 (sai) → Print 2 nghiệm → End

Bộ test case

TC	a	b	c	delta	a=0	delta <0	delta =0	Đường	Kết quả mong đợi
TC1	0	1	1	—	T	—	—	Đ1	Print error
TC2	1	1	1	−3	F	T	—	Đ2	no results
TC3	1	2	1	0	F	F	T	Đ3	x = −1
TC4	1	5	4	9	F	F	F	Đ4	x1 = −4, x2 = −1

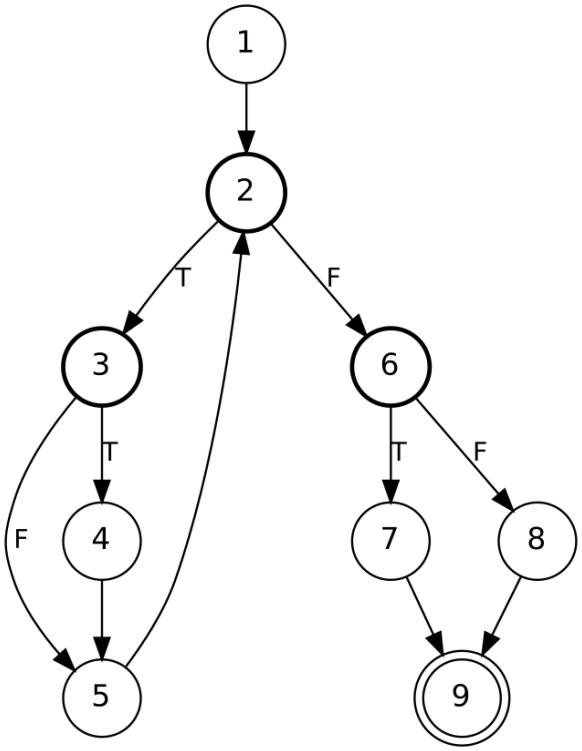
Giải thích mức bao phủ: các điều kiện trong lưu đồ đều là điều kiện đơn (không có &&, ||), nên bao phủ nhánh, bao phủ điều kiện, bao phủ điều kiện/quyết định và bao phủ đa điều kiện trùng nhau. Bộ 4 test case trên cho mỗi điều kiện nhận cả giá trị T lẫn F và đi qua **toàn bộ 4 đường đi** (bao phủ đường đi 100%), nên thỏa bao phủ cấp 4 theo mọi cách phân cấp thường dùng.

Bài 3. Tính trung bình cộng các số dương trong mảng

```
int processArray(int arr[], int n) {
1.  int i = 0;
2.  int count = 0;
3.  int sum = 0;
4.  while (i < n) {              // Decision node 1
5.      if (arr[i] > 0) {        // Decision node 2
6.          sum += arr[i];
7.          count++;
8.      }
9.      i++;
10. }
11. if (count > 0) {            // Decision node 3
12.     return sum / count;
13. } else {
14.     return 0;
15. }
16. }
```

1) CFG và độ phức tạp chu trình V(G)

Nút	Dòng lệnh	Nội dung
1	1–3	Khởi tạo i, count, sum
2	4	while (i < n) — quyết định 1
3	5	if (arr[i] > 0) — quyết định 2
4	6–7	sum += arr[i]; count++
5	9	i++ (quay lại nút 2)
6	11	if (count > 0) — quyết định 3
7	12	return sum / count
8	14	return 0
9	16	Nút thoát (Exit)



Tính V(G) (E = 11 cạnh, N = 9 nút, P = 3 nút quyết định):
• V(G) = E − N + 2 = 11 − 9 + 2 = **4**
• V(G) = P + 1 = 3 + 1 = **4**
• Số vùng của đồ thị = 4 (3 vùng kín + 1 vùng ngoài) = **4**

2) Các đường đi độc lập cơ sở

Đường	Dãy nút	Mô tả
P1	1-2-6-8-9	Không vào vòng lặp (n ≤ 0), count = 0 → return 0
P2	1-2-3-5-2-6-8-9	Lặp 1 lần với phần tử không dương, count = 0 → return 0
P3	1-2-3-4-5-2-6-7-9	Lặp 1 lần với phần tử dương, count > 0 → return sum/count
P4	1-2-6-7-9	**Không khả thi**: bỏ qua vòng lặp thì count = 0 nên không thể rẽ sang nút 7

Có 4 đường độc lập (bằng V(G)) nhưng chỉ 3 đường thực thi được; P4 là đường không khả thi nên không có test case. Để kiểm tra thêm việc lặp nhiều lần, bổ sung đường P5 (phụ thuộc tuyến tính, không thuộc tập cơ sở): 1-2-3-4-5-2-3-5-2-6-7-9.

3) Test case cho từng đường cơ sở

TC	Đường	arr	n	Kết quả mong đợi
TC1	P1	[ ] (rỗng)	0	0
TC2	P2	[−3]	1	0
TC3	P3	[4]	1	4
—	P4	không khả thi	—	—
TC4 (bổ sung)	P5	[6, −2]	2	6

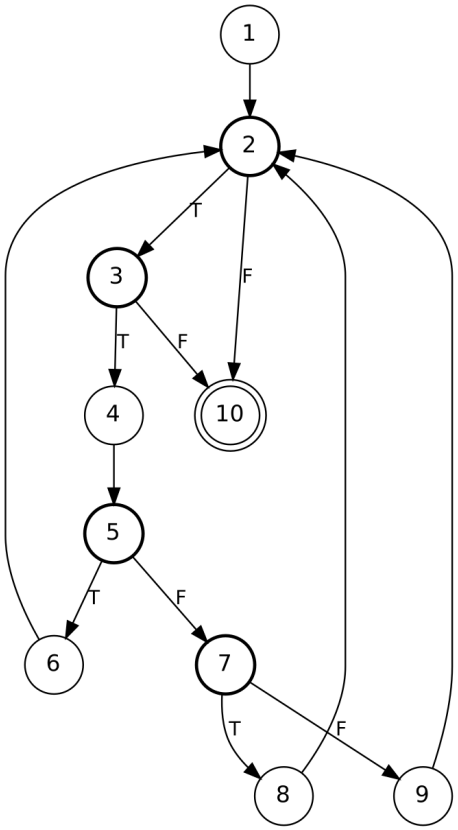
Bài 4. Tìm kiếm nhị phân

```
1.  public int binarySearch(int sortedArray[], int searchValue) {
2.      int bottom = 0;
3.      int top = sortedArray.length - 1;
4.      int middle, locationOfsearchValue;
5.      boolean found = false;
6.      locationOfsearchValue = -1;
8.      while (bottom <= top && !found) {
9.          middle = (top + bottom) / 2;
11.         if (searchValue == sortedArray[middle]) {
12.             found = true;
13.             locationOfsearchValue = middle;
15.         } else if (searchValue < sortedArray[middle]) {
16.             top = middle - 1;
18.         } else {
19.             bottom = middle + 1;
20.         }
21.     }
22.     return locationOfsearchValue;
23. }
```

1) CFG và độ phức tạp chu trình V(G)

Điều kiện của while là điều kiện ghép (**bottom <= top && !found**) nên được tách thành 2 nút quyết định riêng:

Nút	Dòng lệnh	Nội dung
1	2–6	Khởi tạo bottom, top, found, locationOfsearchValue
2	8	bottom <= top — quyết định
3	8	!found — quyết định
4	9	middle = (top + bottom) / 2
5	11	searchValue == sortedArray[middle] — quyết định
6	12–13	found = true; locationOfsearchValue = middle
7	15	searchValue < sortedArray[middle] — quyết định
8	16	top = middle − 1
9	19	bottom = middle + 1
10	22	return locationOfsearchValue (nút thoát)



Tính V(G) (E = 13 cạnh, N = 10 nút, P = 4 nút quyết định):

• V(G) = E − N + 2 = 13 − 10 + 2 = **5**

• V(G) = P + 1 = 4 + 1 = **5**

Lưu ý: nếu coi điều kiện ghép của while là một nút quyết định duy nhất thì V(G) = 4 và cả 4 đường cơ sở đều khả thi; bộ test TC1–TC4 bên dưới vẫn dùng được cho cả hai cách tính.

2) Các đường đi độc lập cơ sở

Đường	Dãy nút	Mô tả
P1	1-2-10	Mảng rỗng: bottom > top ngay từ đầu → return −1
P2	1-2-3-4-5-6-2-3-10	Tìm thấy ngay ở lần lặp đầu → found = true, thoát vòng lặp
P3	1-2-3-4-5-7-8-2-10	searchValue < phần tử giữa → top giảm, hết đoạn tìm → return −1
P4	1-2-3-4-5-7-9-2-10	searchValue > phần tử giữa → bottom tăng, hết đoạn tìm → return −1
P5	1-2-3-10	**Không khả thi**: lúc bắt đầu found = false nên !found luôn đúng

3) Test case cho từng đường cơ sở

TC	Đường	sortedArray	searchValue	Kết quả mong đợi
TC1	P1	[ ] (rỗng)	5	−1
TC2	P2	[5]	5	0
TC3	P3	[5]	3	−1
TC4	P4	[5]	7	−1
—	P5	không khả thi	—	—
TC5 (bổ sung)	1-2-3-4-5-7-8-2-3-4-5-7-9-2-3-4-5-6-2-3-10	[1, 3, 5, 7, 9]	3	1

TC5 là test bổ sung (không thuộc tập cơ sở) đi qua cả ba nhánh trong thân vòng lặp ở nhiều lần lặp liên tiếp.

Bài 5. Hàm tính tiền điện lũy tiến (Def-Use)

```
def calculate_power_bill(usage):
    rate = 0                              # d1: def rate
    total = 0                             # d2: def total
    if usage <= 50:                       # u1: use usage
        rate = 1678                       # d3: def rate
    elif usage <= 100:                    # u2: use usage
        rate = 1734                       # d4: def rate
        total = 50 * 1678                 # d5: def total
    else:
        rate = 2014                       # d6: def rate
        total = 50 * 1678 + 50 * 1734     # d7: def total

    total += (usage - 50) * rate          # u3: usage, u4: rate, u5: total, d8: def total
    return total                          # u6: use total
```

1) Các cặp Định nghĩa – Sử dụng (Def-Use) của biến rate

Biến **rate** được định nghĩa tại d1, d3, d4, d6 và chỉ được sử dụng tại **u4**.

Cặp	Def → Use	Đường def-clear	Nhận xét
(d1, u4)	rate = 0 → u4	Không có	Mọi nhánh (if / elif / else) đều gán lại rate trước u4 nên d1 bị "kill": cặp không khả thi (định nghĩa thừa)
(d3, u4)	rate = 1678 → u4	d3 → d8 (nhánh usage ≤ 50)	Khả thi
(d4, u4)	rate = 1734 → u4	d4 → d8 (nhánh 50 < usage ≤ 100)	Khả thi
(d6, u4)	rate = 2014 → u4	d6 → d8 (nhánh usage > 100)	Khả thi

Kết luận: có 3 cặp khả thi là (d3,u4), (d4,u4), (d6,u4).

2) Bộ test case bao phủ All-Defs cho biến rate

All-Defs: mỗi định nghĩa của rate phải đến được ít nhất một lần sử dụng. d3, d4, d6 lần lượt cần một test; d1 không thể đến bất kỳ lần sử dụng nào nên không thể phủ (bị loại).

TC	usage	Định nghĩa được phủ	Cặp phủ	total mong đợi (theo mã)
TC1	50	d3	(d3, u4)	0 + (50 − 50)·1678 = 0
TC2	75	d4	(d4, u4)	83 900 + 25·1734 = 127 250
TC3	150	d6	(d6, u4)	170 600 + 100·2014 = 372 000

3) Bộ test case bao phủ All-Uses cho biến rate

All-Uses: mỗi cặp (định nghĩa, sử dụng) khả thi phải được phủ ít nhất một lần. Vì rate chỉ có một lần sử dụng (u4) nên 3 cặp khả thi (d3,u4), (d4,u4), (d6,u4) chính là 3 yêu cầu cần phủ, và bộ test thu được **trùng với All-Defs**:

TC	usage	Cặp (Def, Use) được phủ	Đường đi
TC1	50	(d3, u4)	d1-d2-u1-d3-u3/u4/u5/d8-u6
TC2	75	(d4, u4)	d1-d2-u1-u2-d4-d5-u3/u4/u5/d8-u6
TC3	150	(d6, u4)	d1-d2-u1-u2-d6-d7-u3/u4/u5/d8-u6

Nhận xét thêm khi phân tích mã: (1) d1 (rate = 0) là phép gán thừa vì luôn bị ghi đè. (2) Công thức ở nhánh usage > 100 dùng (usage − 50)·2014 thay vì (usage − 100)·2014 nên tính dư tiền (với usage = 150, đúng phải là 271 300 chứ không phải 372 000). (3) Nhánh usage ≤ 50 dùng (usage − 50)·1678 nên cho kết quả âm khi usage < 50 (đúng phải là usage·1678). Đây là các lỗi logic mà bộ test trên có thể phát hiện khi so sánh với kết quả mong đợi theo đúng biểu giá.